Wilkinson's Grammar of Graphics: Theory

Workshop 2 — Module 1

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

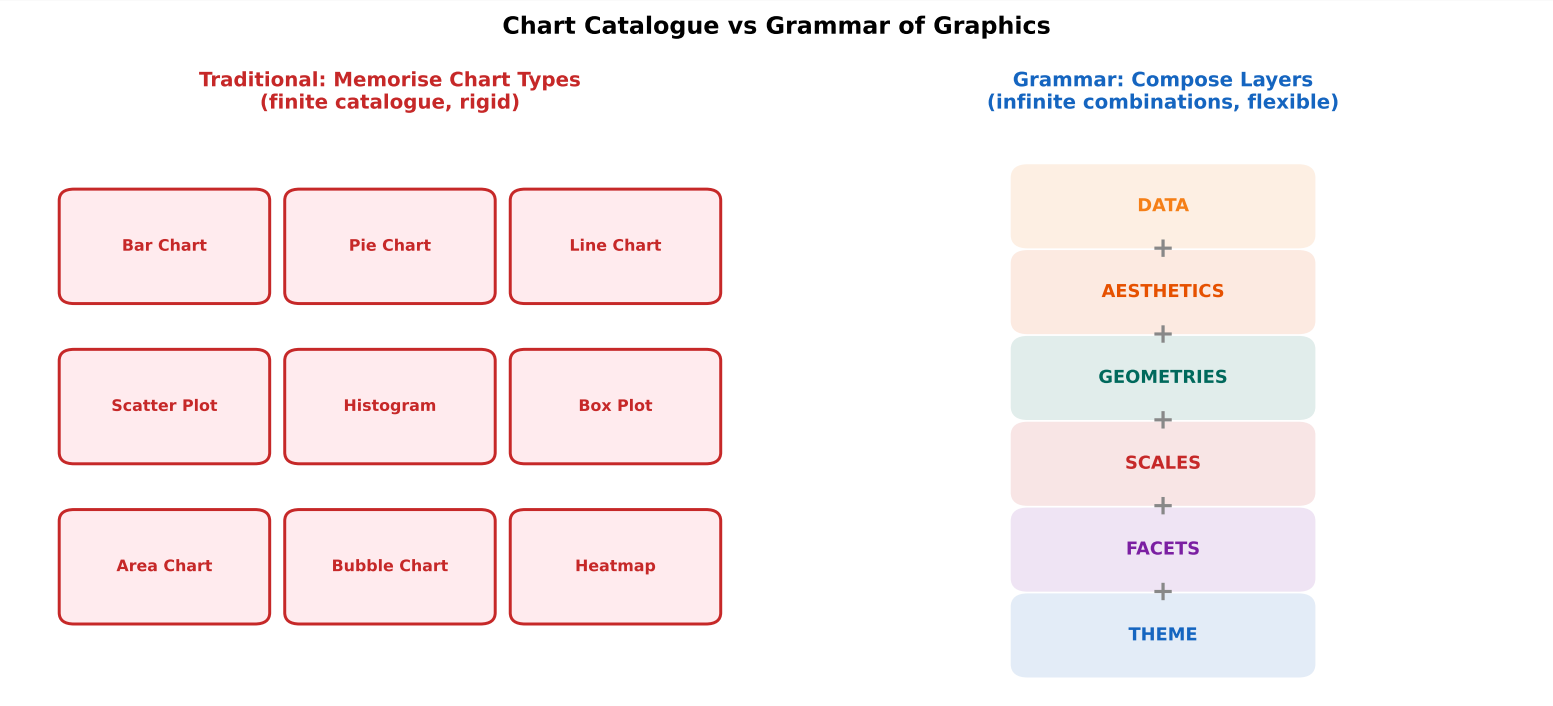
Module Roadmap

1. The problem with chart catalogues
2. Wilkinson's seven-layer specification
3. The algebra of graphics: data $\times$ aesthetics $\times$ geometry
4. Layered construction: building plots incrementally
5. Wilkinson (1999) vs Wickham (2010): theory to implementation

Learning Outcome
You will understand the Grammar of Graphics as a *compositional system* rather than a chart catalogue, and be able to decompose any chart into its constituent layers.

The Problem with Chart Catalogues

Traditional Approach: Memorise Chart Types



Why Catalogues Fail

The catalogue approach:

- Finite list of named chart types
- Each type has its own API/function
- Novel combinations require new types
- No systematic way to modify one aspect

The grammar approach:

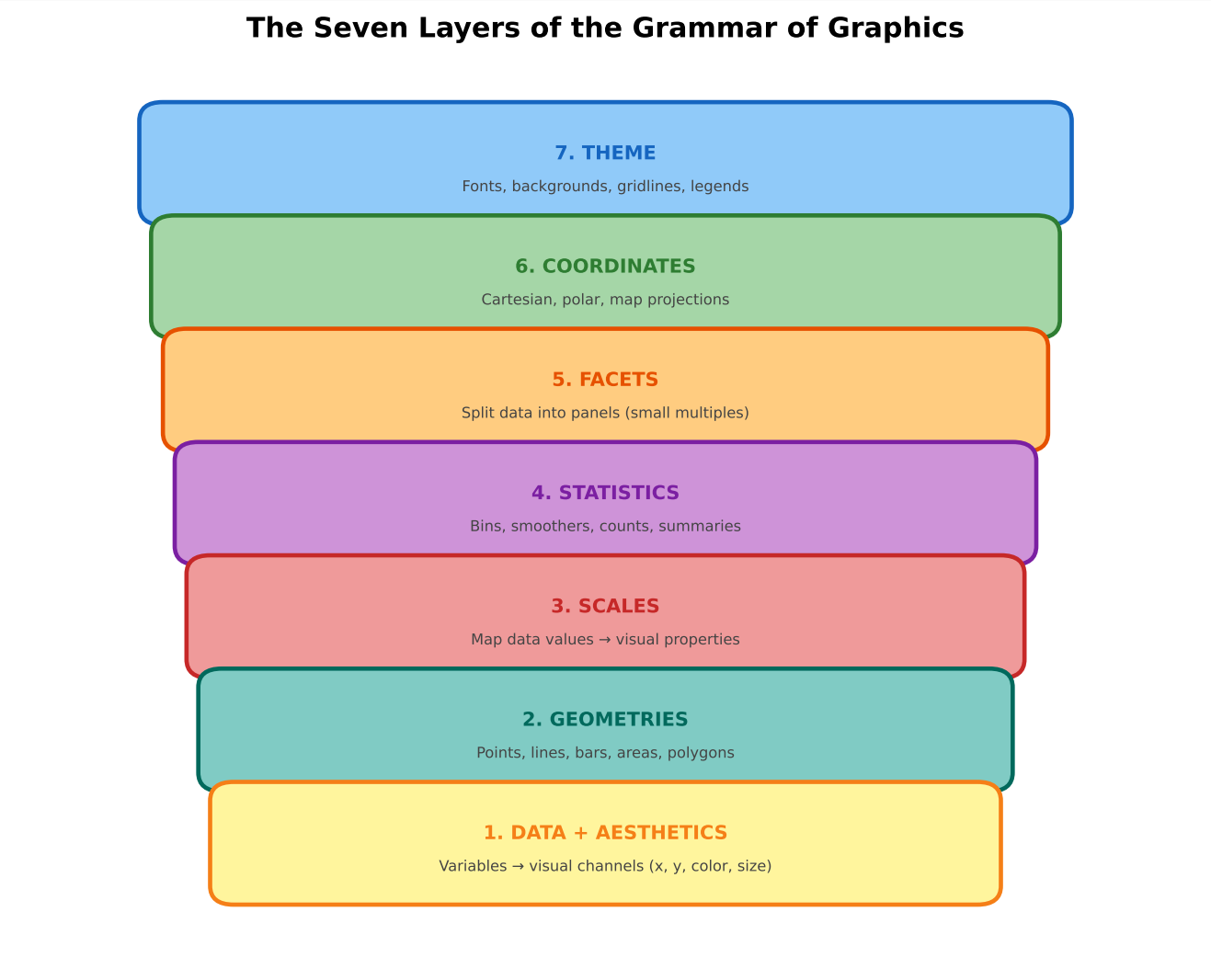
- Small set of composable components
- Components combine freely with "+"
- Novel charts emerge naturally
- Change one layer without rebuilding

Data Insight

A grammar generates an *infinite* set of charts from a finite set of rules — exactly as a natural language grammar generates infinite sentences from finite vocabulary and syntax.

The Seven Layers

Wilkinson's Seven-Layer Specification



Layer 1: Data + Aesthetics

The foundation: map **data variables** to **visual channels**.

Variable	Channel
GDP	x position
Life Expectancy	y position
Continent	colour (hue)
Population	size
Year	animation frame

Pro-Tip
The aesthetic mapping is the single most important decision. A chart is only as good as its variable-to-channel assignment. Revisit Cleveland & McGill (W01-M03) to choose the best channel.

Layer 2: Geometries

Geometries are the visual marks that represent data:

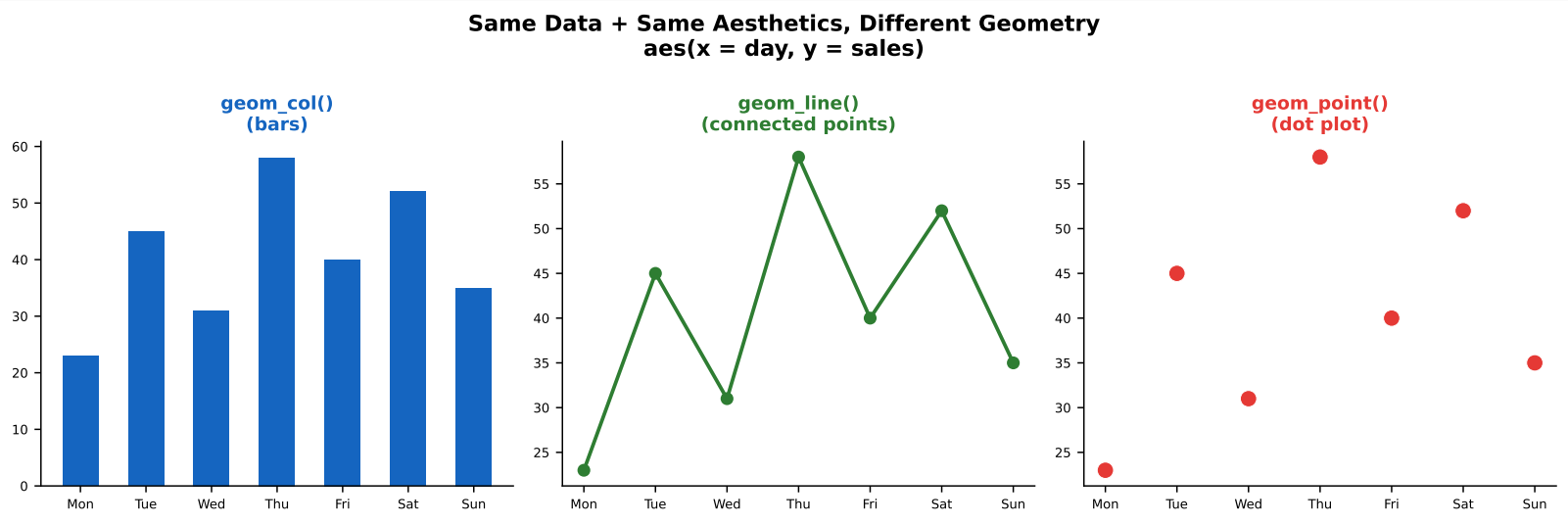
Geom	Mark	Best For
`geom_point`	Circle/dot	Relationships (scatter)
`geom_line`	Connected line	Temporal trends
`geom_col/bar`	Rectangle	Comparisons
`geom_histogram`	Binned rect.	Distributions
`geom_boxplot`	Box + whisker	Group summaries
`geom_area`	Filled region	Cumulative / stacked
`geom_tile`	Coloured cell	Heatmaps

Data Insight

The geometry is *not* the chart type — it is one layer of the chart.

Multiple geometries can be stacked: `geom_point() + geom_smooth()` is two layers on one axes.

Same Data, Different Geometry



Pro-Tip

The data and aesthetic mapping are identical across all three panels. Only the geometry changes. This is the power of the grammar: swap one layer without rewriting anything else.

Layers 3–5: Scales, Statistics, Facets

3. Scales

Map data values to visual properties:

- Continuous → position
- Discrete → colour hue
- Log transform
- Date formatting

4. Statistics

Transform data before rendering:

- `stat_bin` (histogram)
- `stat_smooth` (LOESS)
- `stat_summary` (mean)
- `stat_count` (bar)

5. Facets

Split data into panels by a categorical variable:

- `facet_wrap`
- `facet_grid`
- Shared or free scales
- Tufte's small multiples

Layers 6–7: Coordinates & Themes

6. Coordinate System

Defines the geometry of the plane:

- `coord_cartesian` (default)
- `coord_flip` (swap x/y)
- `coord_polar` (pie, coxcomb)
- `coord_sf` (map projection)

A bar chart in Cartesian is bars; the same bar chart in polar is a pie chart. Only the coordinate system changed.

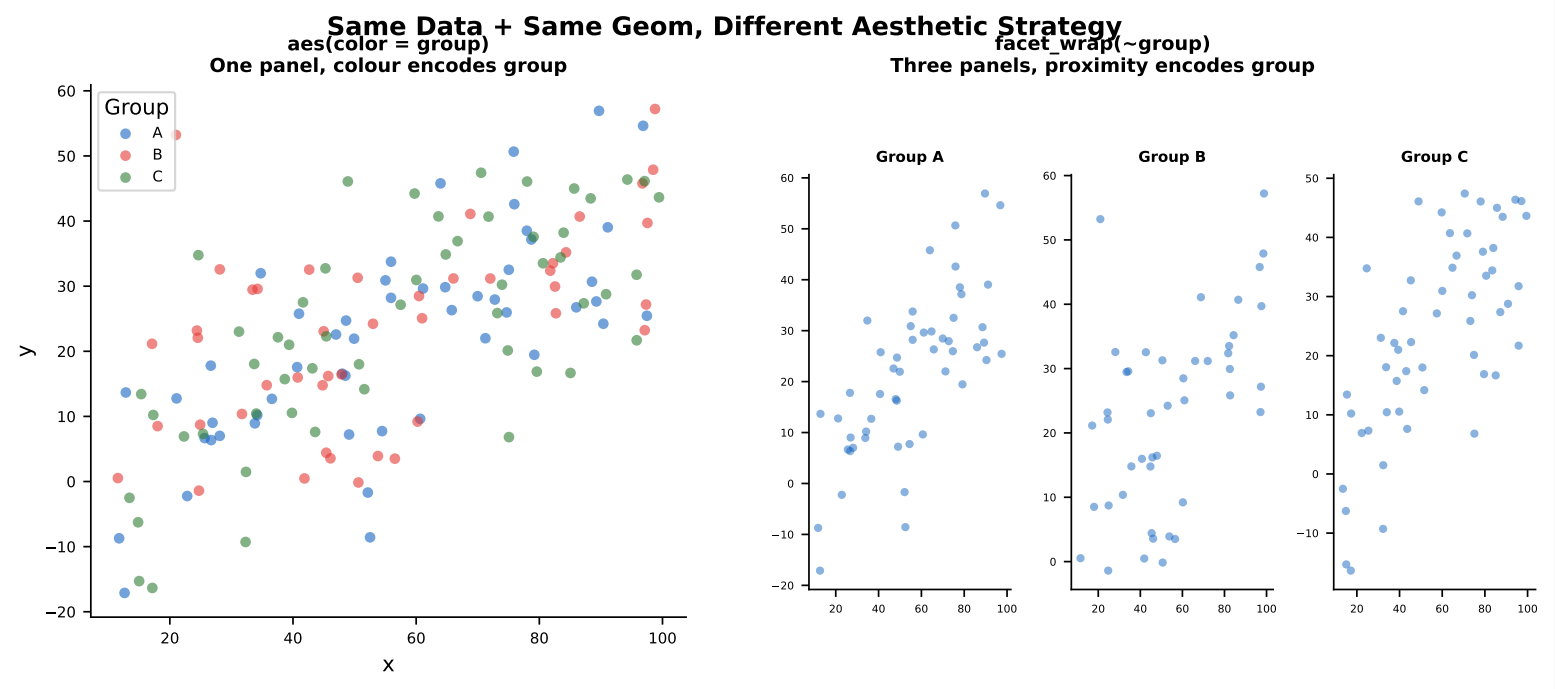
7. Theme

Controls all non-data visual elements:

- Fonts: family, size, weight
- Backgrounds: panel, plot, strip
- Gridlines: major, minor
- Legend: position, box, key

Themes are separable from the data encoding — change the theme without touching the analysis.

Same Data, Different Aesthetic Strategy

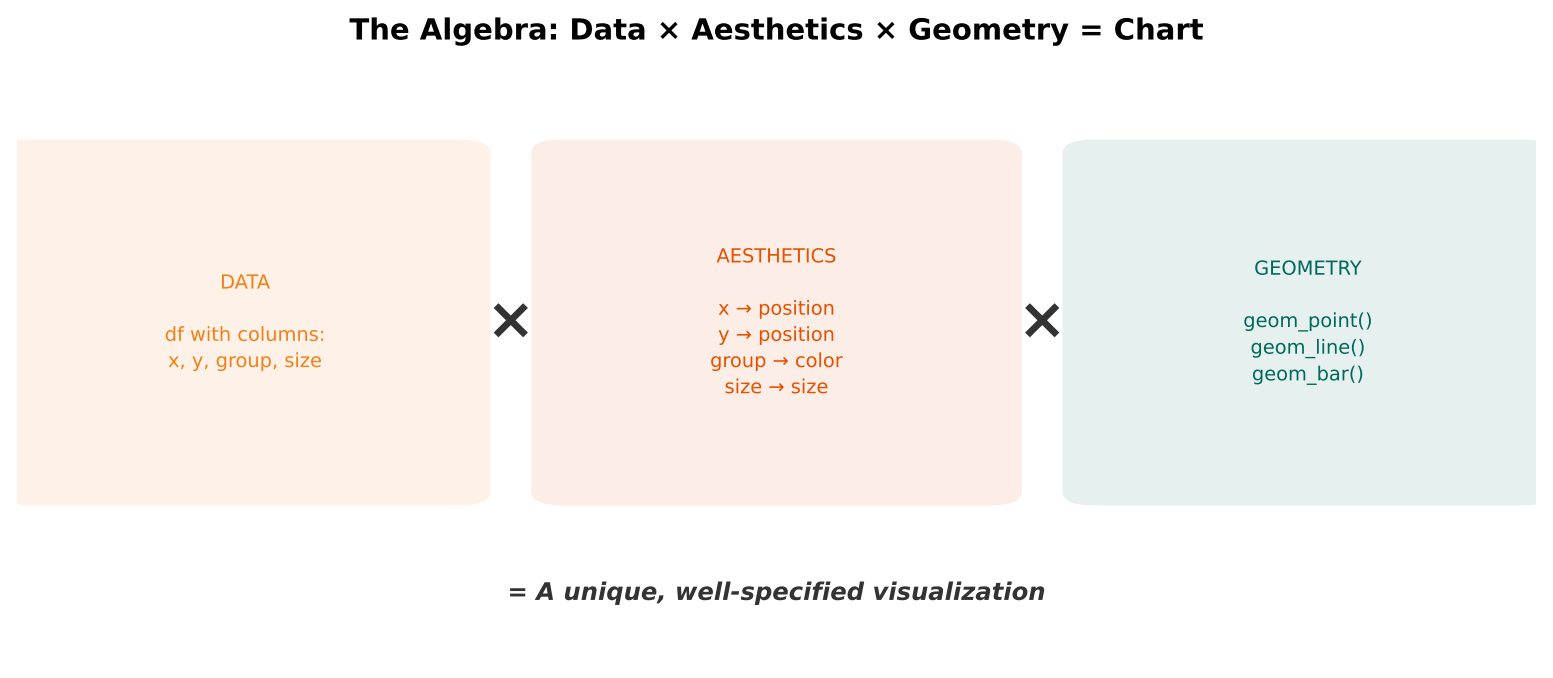


Data Insight

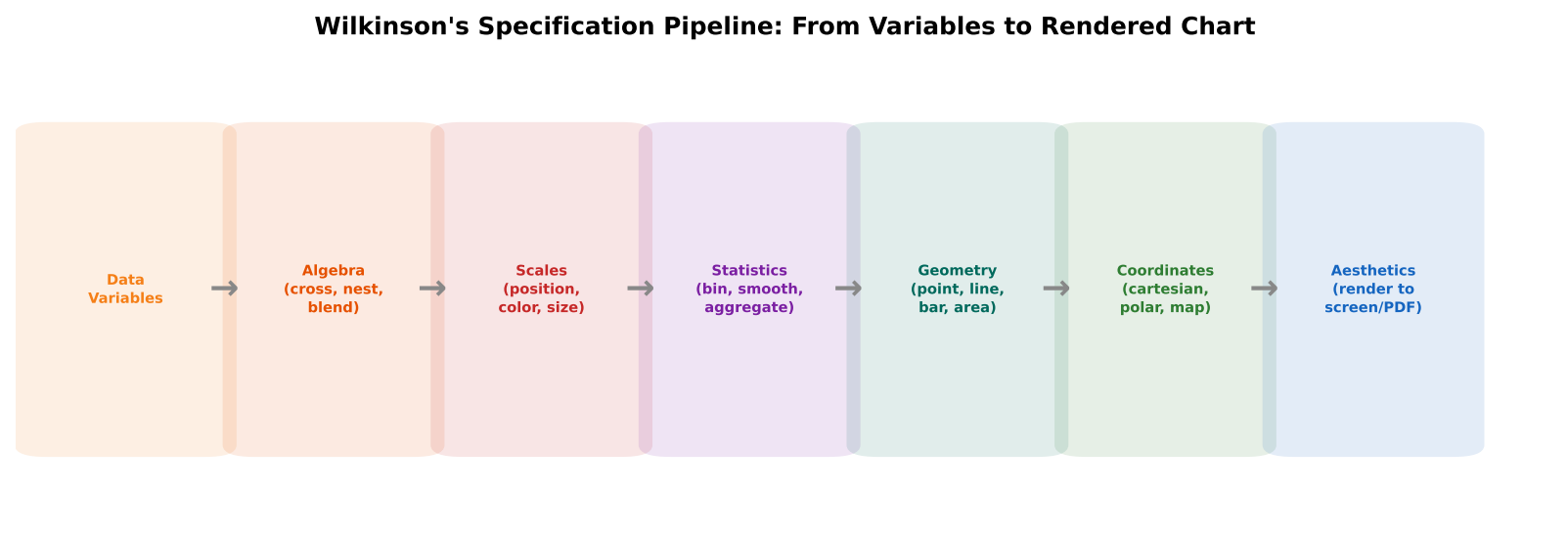
Colour and faceting are both valid ways to encode a grouping variable. Colour keeps all data on one panel (good for ≤ 3 groups); faceting separates panels (better for >3 groups or when groups overlap heavily).

The Algebra of Graphics

Data × Aesthetics × Geometry = Chart



Wilkinson's Specification Pipeline

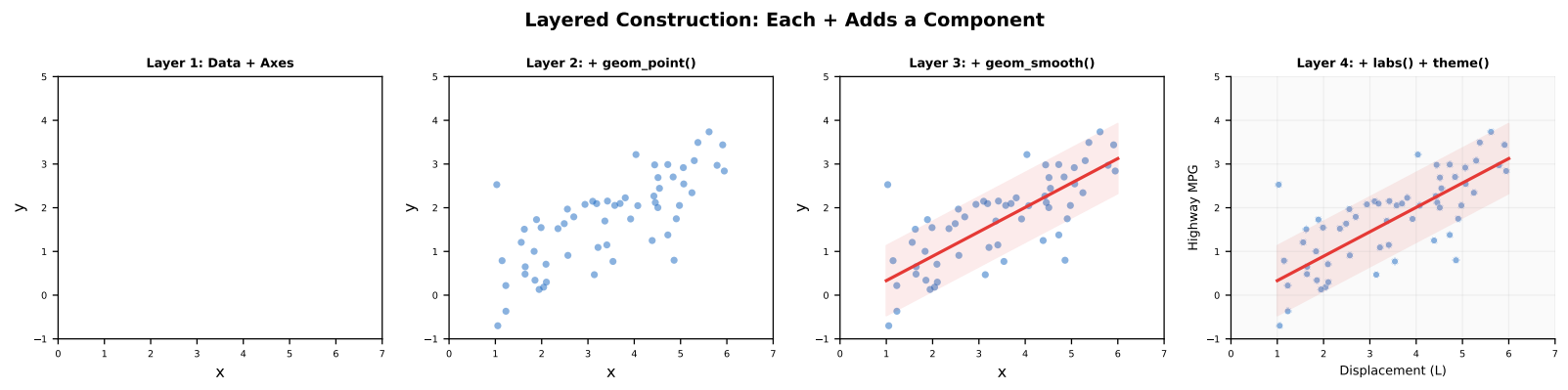


Data Insight

Wilkinson's pipeline is a formal specification: Variables $\rightarrow$ Algebra $\rightarrow$ Scales $\rightarrow$ Statistics $\rightarrow$ Geometry $\rightarrow$ Coordinates $\rightarrow$ Aesthetics (rendering). Each step is independent and can be modified without affecting the others.

Layered Construction

Building a Plot Layer by Layer

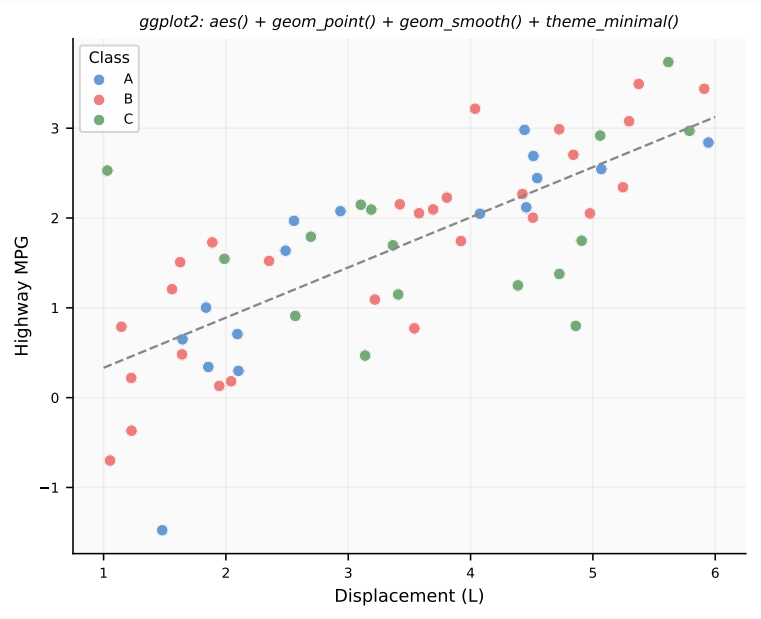


Pro-Tip

Start with data + axes (see the variable ranges), add geometry (see the data), add statistics (see the pattern), then polish with labels and theme. This "progressive reveal" mirrors the design process from W01-M06.

Layered Construction in R (ggplot2)

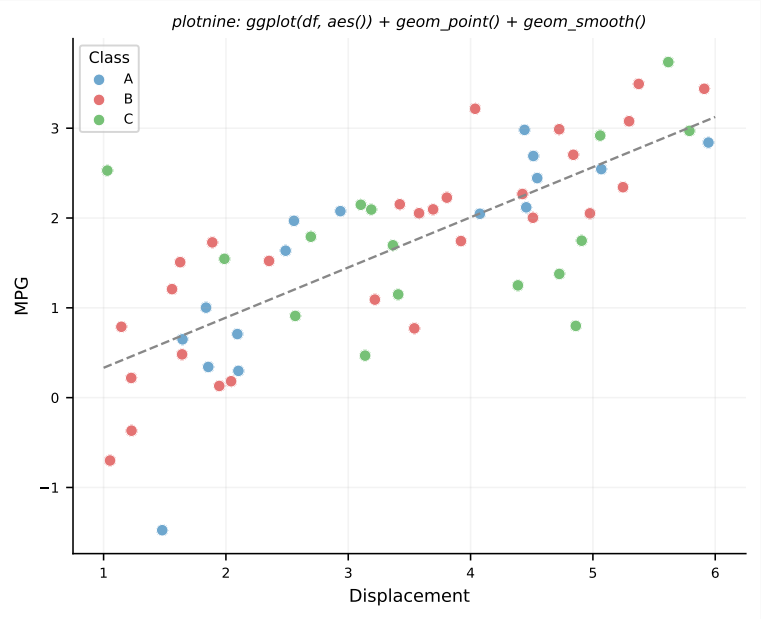
```r
library(ggplot2)

# Layer 1: data + aesthetics
ggplot(mpg,
  aes(x = displ, y = hwy,
      color = class)) +

# Layer 2: geometry
  geom_point(size = 2,
             alpha = 0.6) +

# Layer 3: statistic
  geom_smooth(method = "lm",
              se = TRUE,
              color = "grey40") +

# Layer 4: scale
  scale_color_brewer(
    palette = "Set2") +

# Layer 5: labels + theme
  labs(title = "Displacement vs MPG
       ",
       x = "Engine (L)",
       y = "Highway MPG") +
  theme_minimal(base_size = 11)
```



Each + adds one layer. The order of layers determines the drawing order (first layer drawn first, behind later layers).

Layered Construction in Python (plotnine)

```python
from plotnine import *
import pandas as pd

# plotnine uses identical syntax
# to R's ggplot2

(
  ggplot(mpg,
    aes(x="displ", y="hwy",
        color="class"))

  # Layer 2: geometry
  + geom_point(size=2, alpha=0.6)

  # Layer 3: statistic
  + geom_smooth(method="lm",
                se=True,
                color="grey")

  # Layer 4: scale
  + scale_color_brewer(
      type="qual",
      palette="Set2")

  # Layer 5: labels + theme
  + labs(title="Displacement vs MPG
```



plotnine is a near-exact port of ggplot2 to Python. The only syntax difference: strings for column names instead of bare symbols, and + at the start of each line (Python line continuation).

Wilkinson vs Wickham

Theory (1999) vs Implementation (2010)

Wilkinson (1999) vs Wickham (2010): Theory → Implementation

Dimension	Wilkinson (1999)	Wickham (2010)
Published	1999 (book)	2010 (JCGS paper)
Scope	General theory (any software)	R implementation (ggplot2 package)
Layers	7 theoretical components	7 practical layers (+ position adj.)
Implementation	Conceptual specification	Working R code with + operator
Statistics	Explicit transformation step	Implicit via stat_*() functions
Audience	Researchers, theory	Practitioners, applied

Wickham's Key Contributions

Wickham's 2010 paper and the `ggplot2` package extended Wilkinson's theory with four practical innovations:

1. **Layered architecture**: multiple geom/stat pairs stacked on a single axes via +, each with its own aesthetic mapping if needed
2. **Position adjustments**: `dodge`, `stack`, `fill`, `jitter` — modify the placement of marks without changing the geometry
3. **Default statistics**: each geom has a default stat (`geom_bar` defaults to `stat_count`), reducing boilerplate
4. **Theme system**: a complete separation of data encoding from visual styling, enabling one-line theme swaps

Pro-Tip
Read Wickham (2010) as the "user manual" for the grammar. Read Wilkinson (1999/2005) for the "why" behind the design.

Synthesis

Decomposing Any Chart into Grammar Components

Exercise: decompose a grouped bar chart into its layers.

Layer	**Specification**
Data	sales data frame (product, quarter, revenue)
Aesthetics	x = quarter, y = revenue, fill = product
Geometry	`geom_col(position = "dodge")`
Scale	`scale_fill_brewer(palette = "Set2")`
Statistics	identity (raw values, no transformation)
Facets	none (single panel)
Coordinates	Cartesian (default)
Theme	`theme_minimal()`

Changing any single row produces a different chart. Change geometry to `geom_line()` → line chart. Change coordinates to `coord_polar()` → radial bar. Change facets to `facet_wrap(~product)` → small multiples.

Key Takeaways

1. The Grammar of Graphics replaces a **chart catalogue** with a **compositional system**: data + aesthetics + geometry + scales + statistics + facets + coordinates + theme.
2. The **algebra** Data $\times$ Aesthetics $\times$ Geometry generates infinite chart types from finite components.
3. **Layered construction** builds plots incrementally: each + adds one component. Start with data, end with theme.
4. The same data and aesthetics with a **different geometry** produce a different chart — swap one layer, keep the rest.
5. **ggplot2** (R) and **plotnine** (Python) implement the grammar with near-identical syntax.

Essential Readings for This Module

- Wilkinson, L. (2005). *The Grammar of Graphics*, 2nd ed., Chapters 1–2 (Introduction, How to Make a Pie).
- Wickham, H. (2010). "A Layered Grammar of Graphics." *Journal of Computational and Graphical Statistics*, 19(1), 3–28.
- Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis*, 2nd ed., Chapter 1 (Getting Started).
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*, Chapter 2 (Visualizing Data: Mapping Data onto Aesthetics).

Next Module: Wickham's Layered Grammar & ggplot2 (W02-M02)